

Agentic RAG

Classic RAG searches once and hopes for the best.

Agentic RAG plans, searches, checks, and searches again.

RAG = Retrieval-Augmented Generation: the LLM looks things up before it answers.

Classic RAG



one search, one answer

Agentic RAG



plan, retrieve, verify, repeat

Swipe →

Give the LLM your documents

An LLM only knows its training data. RAG (Retrieval-Augmented Generation) looks up relevant text first, then answers from it.

? User question

"What is our refund policy for damaged items?"



Retrieve

Embed the question, find the 3 most similar chunks in a vector database



Generate

LLM writes the answer using those chunks as context

Works well for simple questions with one clear answer in one place.

Where classic RAG breaks

Question: "Which of our two suppliers had more late deliveries last quarter, and why?"



 Top 3 chunks retrieved: **all about Supplier A.**
Delivery counts live in a SQL table, not in the documents.



 "Supplier A had several delays due to port congestion."
Half an answer, stated with full confidence.

- ❌ One search, even when the question has two parts
- ❌ One source only (no SQL, no web, no API)
- ❌ Nothing checks if the retrieved text was relevant

Put an agent in the loop

An LLM agent controls retrieval. It decides **what** to search, **where**, and **when** to stop.

1



Plan

Break the question into sub-questions

2



Route

Pick a tool: vector DB, SQL, web search, API

3



Retrieve

Run the search or query

4



Grade

Is this relevant and enough? If not, rewrite the query and go back to 2



5



Answer + self-check

Write the answer, verify it against the sources, cite them

What the agent actually does

"Which supplier had more late deliveries last quarter, and why?"

1

Plan: (a) count delays per supplier (b) find reasons for each

2

SQL tool route

```
SELECT supplier, COUNT(*) ... WHERE late=1
```

→ A: 14 · B: 23

3

Vector search "late delivery reasons Supplier B" → incident reports

4

Grade: reasons for B found, reasons for A missing → **rewrite query**  → found

5

Answer: "Supplier B, 23 vs 14. B: customs holds. A: port congestion." + sources

Illustrative example (numbers are made up)

5 patterns from research

1

Routing

Simple question → answer directly. Hard one → multi-step retrieval.

Adaptive-RAG, Jeong et al. 2024

2

Corrective retrieval

Grade the retrieved docs. Poor quality → fall back to web search.

CRAG, Yan et al. 2024

3

Self-reflection

Model decides when to retrieve and critiques its own output.

Self-RAG, Asai et al. 2023

4

Reason + Act loop

Think, call a tool, observe, repeat.

ReAct, Yao et al. 2022

5

Multi-agent

Separate agents for SQL, docs and web, run by one coordinator.

QUICK COMPARISON

Classic vs Agentic

	Classic RAG	Agentic RAG
Searches	One, fixed	As many as needed
Sources	One vector DB	Vector DB, SQL, web, APIs
Query	Used as typed	Split and rewritten
Checks results	No	Yes, grades and retries
Speed & cost	Fast, cheap	Slower, more LLM calls
Best for	FAQ, simple lookups	Multi-part, multi-source questions

Don't make everything agentic. If one search answers 90% of your questions, classic RAG is the right tool.

BEFORE YOU BUILD

Keep in mind

- ✓ Cap the loop (e.g. 3 retries), or the agent can search forever
- ✓ Log every step. Debugging an agent without traces is guesswork
- ✓ Evaluate retrieval separately from the final answer
- ✓ More steps = more latency and cost. Measure both

Papers to read

Agentic RAG: A Survey, Singh et al. 2025

arxiv.org/abs/2501.09136

Self-RAG, Asai et al. 2023 · arxiv.org/abs/2310.11511

Corrective RAG, Yan et al. 2024 · arxiv.org/abs/2401.15884

Adaptive-RAG, Jeong et al. 2024 · arxiv.org/abs/2403.14403

ReAct, Yao et al. 2022 · arxiv.org/abs/2210.03629